DEEP LEARNING

Kaggle Competition Report

Binary Image Classification Challenge

RAJA SUDHA SRI HARIKA	CS25MTECH02001
Deeba Afridi	CS24MTECH12018
Rajat Maheswari	CS25MTECH02007

Contents

1 Introduction

1.1 Problem Statement

This report presents the methodology and findings of our team's submission to the Kaggle binary image classification competition, undertaken as part of the Deep Learning subject assessment.

The task is to classify images into two categories:

- **Class 0** — Negative (no distinguishing pattern)
- **Class 1** — Positive (contains at least one sphere *and* at least one cube)

Model performance is evaluated using **classification accuracy** on a held-out test set.

1.2 Dataset Analysis

The dataset consists of rendered 3D scenes, each image containing exactly **four objects**. Each object varies along the following dimensions:

- **Shape:** cube, sphere, or cylinder
- **Material:** matte or metallic
- **Size:** objects may differ in size within an image
- **Position/Occlusion:** objects can partially occlude one another

Table 1: Dataset split summary

Split	Class 0	Class 1	Observations
Train	9,000	9,000	Labelled; balanced
Public Test	5,010 (unlabelled)		

Class distinction rule (discovered during EDA): We have perfromed Exploratory Data Analysis(EDA) and it is observed that Class 1 images contain *at least one sphere and at least one cube.*Whereas, Class 0 images do not satisfy this joint condition.

2 Data Analysis

2.1 Dataset Visualization

The following are the observations:

- Every image contains **exactly four objects**.
- Objects belong to one of three shape classes: **cube**, **sphere**, or **cylinder**.
- Each object has one of two surface materials: **matte** or **metallic**.
- Object **sizes are not uniform** — within a single image, objects are of different sizes.
- **Partial occlusion** is present: some objects are partially hidden behind others, reducing the visible surface area.
- Lighting and background are consistent across the dataset (synthetic renders).

2.2 Target Feature Analysis

The following are the observations from Dataset:

- **Class 1 (Positive):** The image contains *at least one sphere* **AND** *at least one cube*, irrespective of material, size, or position.
- **Class 0 (Negative):** The image does *not* satisfy the above condition — it may contain cylinders and/or only one of {sphere, cube}, or neither.

3 Methodology

3.1 Preprocessing & Augmentation

- All input images are resized to 96×96.
- No mean/std normalization is applied to the images.
- Pixel values are divided by 255.0 to scale them to the range $[0, 1]$.
- Each RGB image is converted to grayscale.
- Sobel edge detection is applied in both horizontal and vertical directions.
- The Sobel edge magnitude map is computed from the horizontal and vertical gradients.
- The edge magnitude map is normalized by dividing it by its maximum value.
- Therefore, the final edge image is also scaled to the range $[0, 1]$.
- Effectively, this corresponds to using:

$$\text{mean} = [0], \qquad \text{std} = [1]$$

  with min-max scaling performed per image.
- During training, data augmentation is applied to improve model generalization.
- The following augmentations are used during training:
  - Brightness jitter randomly sampled from $[0.4,\ 1.8]$
  - Contrast jitter sampled from $[0.4,\ 2.0]$
  - Saturation jitter sampled from $[0.0,\ 2.0]$
  - Hue jitter sampled from $[-0.5,\ 0.5]$
  - Random horizontal flip with probability $p = 0.5$
  - Random rotation with angle sampled from $[-15°,\ +15°]$
- During validation and testing, only resizing and conversion to grayscale are applied.
- No training augmentation is applied during validation or testing.

4 Approaches

4.1 Slot Attention Approach

- Slot Attention is an object-centric learning method. Instead of representing the whole image using one single feature vector, the model learns a fixed number of latent vectors called **slots**.
- Each slot is expected to represent one object or one meaningful part of the scene.
- Each image is resized to 64×64 and normalized to the range $[-1, 1]$.
- The model is trained in an unsupervised manner using image reconstruction loss. That means the model is not given object labels or segmentation masks.
- It learns object-wise representations only by trying to reconstruct the input image.
- The main idea is:

$$\text{Image} \rightarrow \text{CNN Encoder} \rightarrow \text{Slot Attention} \rightarrow \text{Decoder} \rightarrow \text{Reconstructed Image}$$

The model first extracts spatial image features using a CNN encoder. These features are then passed to the Slot Attention module. The Slot Attention module assigns different parts of the image to different slots using an attention mechanism. Each slot is then decoded separately into an image reconstruction and a mask. Finally, all slot-wise reconstructions are combined using the masks to obtain the final reconstructed image.

Motivation Behind Slot Attention

Normal CNNs learn a single global representation for the whole image. This is useful for classification, but it does not explicitly separate different objects in the image. For object-centric reasoning, it is useful to represent each object separately.

Slot Attention solves this problem by learning multiple latent vectors called slots:

$$S = \{s_1, s_2, \ldots, s_K\}$$

where K is the number of slots. Each slot can attend to a different region or object in the image.

The motivation behind using Slot Attention is:

- **Object-centric representation:** Each slot can learn to represent one object or one component of the image.

- **Unsupervised object discovery:** The model does not require object labels or segmentation masks. It learns object separation through reconstruction.
- **Permutation invariance:** The order of objects in an image should not matter. Slots provide an unordered set-like representation.

Architecture

The architecture consists of four main components:

1. CNN Encoder
2. Positional Embedding
3. Slot Attention Module
4. Spatial Broadcast Decoder

Input Preprocessing

Each input image is converted into a tensor, resized to 64×64, and rescaled to the range $[-1, 1]$:

$$x \in \mathbb{R}^{3\times64\times64}$$

where 3 represents the RGB channels.

CNN Encoder

The encoder extracts local visual features from the input image. It consists of four convolution layers. Each convolution uses kernel size 5, stride 1, and padding 2, so the spatial resolution remains 64×64.

The hidden dimensions are:

$$(64, 64, 64, 64)$$

Therefore, the encoder converts the input image into a feature map:

$$x \in \mathbb{R}^{3\times64\times64} \rightarrow F \in \mathbb{R}^{64\times64\times64}$$

Here, each spatial location has a 64-dimensional feature vector.

Positional Embedding

CNN features alone may not contain explicit information about spatial position. Therefore, positional embeddings are added to the encoder output.

$$F_{\text{pos}} = F + \text{PosEmbed}(F)$$

Flattening Spatial Features

After positional embedding, the feature map is flattened across spatial dimensions:

$$F_{\text{pos}} \in \mathbb{R}^{64\times64\times64}$$

Since:

$$64 \times 64 = 4096$$

the feature map becomes:

$$F_{\text{flat}} \in \mathbb{R}^{4096\times64}$$

So each image is represented as 4096 feature vectors, one for each spatial position.

Slot Attention Module

The Slot Attention module receives the flattened encoder features and produces a fixed number of slots.

In this implementation:

$$K = 7$$

where K is the number of slots.

Each slot has dimension:

$$D_{\text{slot}} = 64$$

Therefore, the output of Slot Attention is:

$$\text{slots} \in \mathbb{R}^{7\times64}$$

Attention Mechanism

For each iteration, the Slot Attention module computes queries, keys, and values.

The slots are projected into queries:

$$Q = W_q S$$

The input features are projected into keys and values:

$$K = W_k F$$

$$V = W_v F$$

Attention logits are computed as:

$$A = \frac{KQ^T}{\sqrt{D_{\text{slot}}}}$$

Then softmax is applied over the slots:

$$\text{Attn} = \text{softmax}(A)$$

This tells each spatial feature which slot it should belong to.

Slot Update

After attention, each slot receives a weighted sum of input features:

$$U = \text{Attn}^T V$$

where U is the update vector for each slot.

The slots are updated using a GRU cell:

$$S^{t+1} = \text{GRU}(U, S^t)$$

Then an MLP is applied with a residual connection:

$$S^{t+1} = S^{t+1} + \text{MLP}(\text{LayerNorm}(S^{t+1}))$$

This process is repeated for multiple iterations.

After these iterations, the code performs one additional update using detached slots.

Decoder

Each slot is decoded separately. First, each slot vector of size 64 is reshaped and spatially broadcasted

The decoder uses transposed convolutions to upsample each slot representation back to the original image resolution:

$$4 \times 4 \to 8 \times 8 \to 16 \times 16 \to 32 \times 32 \to 64 \times 64$$

For each slot, the decoder outputs 4 channels:

$$\text{Output per slot} \in \mathbb{R}^{4 \times 64 \times 64}$$

The first 3 channels represent the reconstructed RGB image:

$$\text{reconstruction} \in \mathbb{R}^{3 \times 64 \times 64}$$

The last channel represents the mask logits:

$$\text{mask} \in \mathbb{R}^{1 \times 64 \times 64}$$

Mask Normalization

The masks from all slots are normalized using softmax across the slot dimension:

This ensures that, for every pixel, the masks across all slots sum to 1.

Final Reconstruction

The final reconstructed image is computed as a weighted sum of all slot-wise reconstructions:

$$\hat{x} = \sum_{k=1}^{K} m_k \odot r_k$$

Loss Function

The model is trained using mean squared error reconstruction loss:

$$\mathcal{L} = \text{MSE}(\hat{x}, x)$$

where x is the original input image and $\hat{x}$ is the reconstructed image.

The loss encourages the model to reconstruct the full image correctly. Since the decoder

reconstructs the image using separate slots and masks, the model learns to divide the scene into object-like components.

Hyperparameter	**Value**
Dataset	CLEVR
Input Resolution	64×64
Decoder Initial Resolution	4×4
Number of Slots	7
Maximum Objects per Image	6
Slot Dimension	64
Number of Slot Attention Iterations	3
Encoder Hidden Dimensions	$(64, 64, 64, 64)$
Decoder Hidden Dimensions	$(64, 64, 64, 64)$
Kernel Size	5
MLP Hidden Size in Slot Attention	128
Batch Size	64
Validation Batch Size	64
Optimizer	Adam
Learning Rate	4×10^{-4}
Weight Decay	0.0
Maximum Epochs	100
Learning Rate Scheduler	LambdaLR with warmup and exponential decay
Scheduler Gamma	0.5
Warmup Steps Percentage	0.02
Decay Steps Percentage	0.2
Loss Function	Mean Squared Error Reconstruction Loss
Number of Workers	4
Framework	PyTorch + PyTorch Lightning
Hardware	GPU if available, otherwise CPU

Table 2: Hyperparameters used for the Slot Attention model.

Training Details

The model is trained using the Adam optimizer. The initial learning rate is:

$$4 \times 10^{-4}$$

The learning rate schedule contains two parts. First, the learning rate is linearly warmed up during the initial training steps. Then it decays gradually using an exponential decay factor.

The scheduler factor is computed as:

$$\text{factor} = \gamma^{\frac{\text{step}}{\text{decay steps}}}$$

where:

$$\gamma = 0.5$$

The model is trained for a maximum of 100 epochs.

Thus, even without supervision, the model learns to separate scenes into object-like components.

4.2 Dual CNN

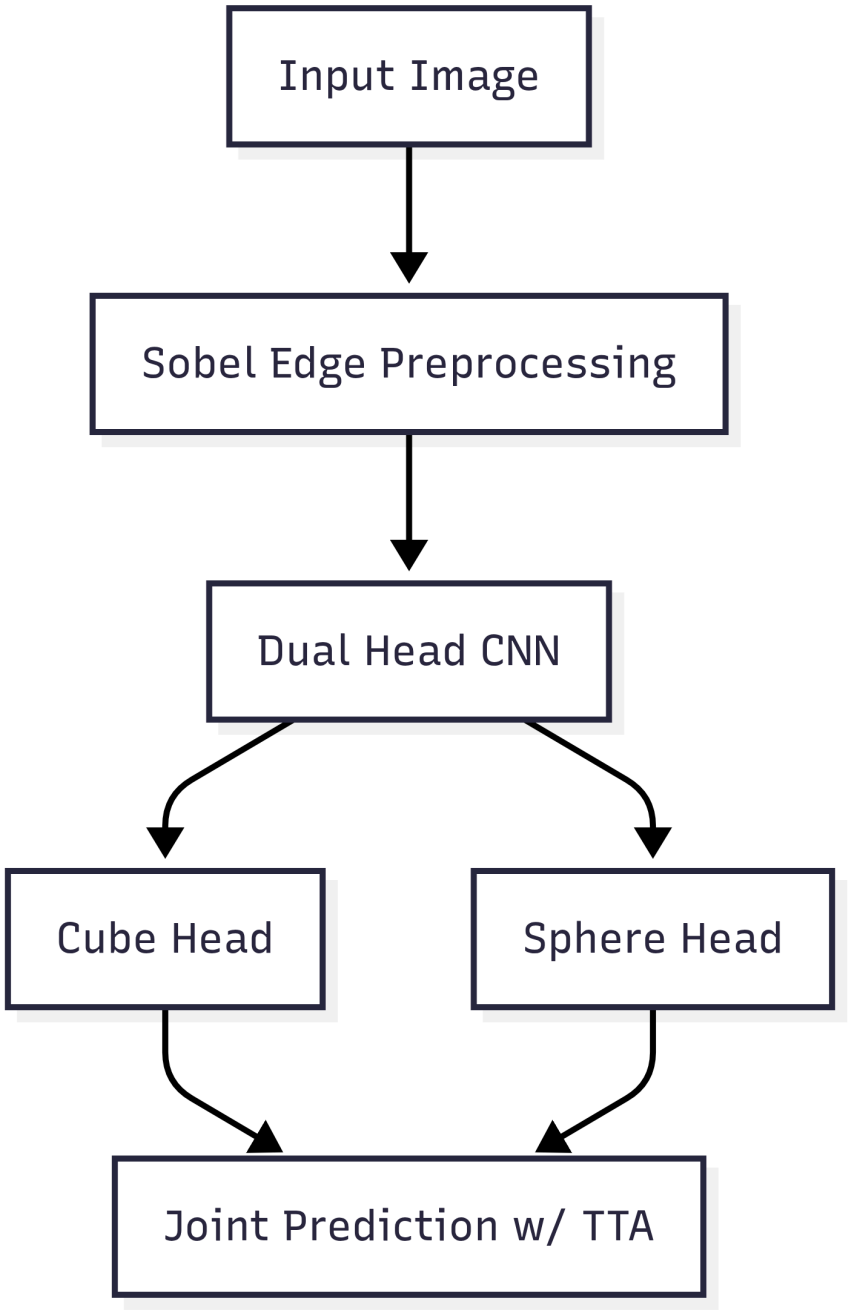


Figure 1: Dual Head CNN

Approach

- The objective of the model is to perform binary image classification.
- This approach uses **Dual Head CNN** for predicting the label of the given image.
- As observed from the dataset that the images can be classified into label 1 if there exist atleast one sphere and one cube
- Hence, The idea is to decompose the final decision into two intermediate semantic predictions: whether a cube is present and whether a sphere is present.
- The final class probability is combination these two probabilities.
- Each input image is first resized to 96×96.
- The RGB image is converted into grayscale and then Sobel edge filters are applied in the horizontal and vertical directions.
- The gradient of image is computed from these Sobel responses.
- This edge-magnitude image highlights object boundaries and shape information, which is useful because the task depends mainly on identifying geometric objects.
- The resulting edge image is stacked into three channels so that it can be passed into a CNN.
- During training, data augmentation is applied to improve generalization.
- The augmentations include random brightness, contrast, saturation, hue changes, horizontal flipping, and small random rotations.
- These transformations make the model more robust to changes in lighting, orientation, and appearance.

The model is trained using a custom dual loss. The two heads predict:

$$p_{\text{cube}} = \sigma(z_{\text{cube}})$$

and

$$p_{\text{sphere}} = \sigma(z_{\text{sphere}})$$

where z_{cube} and z_{sphere} are the logits produced by the two output heads.

The final probability that the image belongs to the positive class is computed as:

$$p_{\text{overall}} = p_{\text{cube}} \times p_{\text{sphere}}$$

Thus, the model predicts the positive class only when both cube and sphere are likely to be present. This is because the +ve prediction depends on the joint presence of two

objects.

Architecture

The **Dual-Head Convolutional Neural Network** model consists of the following main components:

1. **Coordinate Augmentation Layer:** Before applying convolution, two extra coordinate channels are added to the input image. These channels encode the normalized x and y positions of each pixel. Therefore, the CNN receives spatial position information along with image intensity information. This helps the model learn where objects are located in the image.
2. **CNN Stem:** The input is first passed through a convolutional layer followed by instance normalization and ReLU activation. Since the original image has 3 channels, 2 coordinate channels were added, the first convolution receives 5 input channels.
3. **Residual Convolution Blocks:** The network contains multiple convolutional blocks. Each block uses two 3×3 convolution layers with instance normalization and ReLU activation. Residual connections are also added. These connections help in stable training and allow gradients to flow more easily through the network.
4. **Downsampling Layers:** The spatial size of the feature maps is gradually reduced using stride-2 convolutions. At the same time, the number of channels is increased from 32 to 64, 128, 256, and finally 384. This allows the model to learn low-level features in early layers and high-level semantic features in deeper layers.
5. **Global Pooling:** Adaptive max pooling is used to convert the final feature map into a fixed-size feature vector of dimension 384. This allows the model to summarize the most important features from the entire image.
6. **Dropout:** Dropout with probability 0.3 is applied before the output heads to reduce overfitting.
7. **Dual Output Heads:** Two separate linear layers are used:

$$z_{\text{cube}} = W_{\text{cube}} f + b_{\text{cube}}$$

$$z_{\text{sphere}} = W_{\text{sphere}} f + b_{\text{sphere}}$$

where f is the final feature vector. One head predicts cube presence and the other predicts sphere presence.

Motivation Behind Dual-Head CNN

- A normal binary CNN directly predicts whether the image belongs to class 0 or class 1.
- In this problem, the positive class can be interpreted as the joint presence of two visual concepts: cube and sphere. Therefore, instead of learning the final class directly, the model is encouraged to learn two separate object-level concepts.

The motivation behind using a Dual-Head CNN is as follows:

- **Semantic Decomposition:** The final binary decision is decomposed into two meaningful sub-decisions: cube detection and sphere detection.
- **Better Feature Learning:** Since each head focuses on a different object, the shared CNN backbone learns features that are useful for identifying both objects.
- **Joint Probability Modeling:** The final probability is computed as:

$$p_{\text{final}} = p_{\text{cube}} \times p_{\text{sphere}}$$

  This means the final prediction is high only when both heads are confident.
- **Improved Robustness:** If one object is missing, the corresponding probability becomes low, and therefore the final probability also becomes low.
- **Regularization through Auxiliary Heads:** The individual cube and sphere heads act as auxiliary learning signals. This helps the model learn more structured and interpretable representations.

Loss Function

The model uses a custom dual loss. First, the cube and sphere logits are converted into probabilities using sigmoid:

$$p_{\text{cube}} = \sigma(z_{\text{cube}})$$

$$p_{\text{sphere}} = \sigma(z_{\text{sphere}})$$

The joint probability is computed as:

$$p_{\text{both}} = p_{\text{cube}} p_{\text{sphere}}$$

This joint probability is converted back into a logit:

$$z_{\text{both}} = \log\left(\frac{p_{\text{both}}}{1 - p_{\text{both}}}\right)$$

The main loss is binary cross-entropy between the joint logit and the ground truth label:

$$\mathcal{L}_{\text{joint}} = \text{BCEWithLogits}(z_{\text{both}}, y)$$

For positive samples, an additional individual loss is added to encourage both heads to predict positive:

$$\mathcal{L}_{\text{ind}} = \text{BCEWithLogits}(z_{\text{cube}}, y) + \text{BCEWithLogits}(z_{\text{sphere}}, y)$$

The final loss is:

$$\mathcal{L} = \mathcal{L}_{\text{joint}} + 0.5\mathcal{L}_{\text{ind}}$$

Hyperparameter	**Value**
Optimizer	AdamW
Learning Rate	3×10^{-4}
LR Scheduler	CosineAnnealingLR
Batch Size	96
Epochs	25
Loss Function	Custom Dual Binary Cross-Entropy Loss
Weight Decay	1×10^{-4}
Early Stopping Patience	5 epochs
Gradient Clipping	1.0
Input Image Size	96×96
Dropout	0.3
Validation Images per Class	500
Random Seed	42
Number of Workers	0
Hardware	CUDA GPU if available, otherwise CPU
Framework	PyTorch

Table 3: Hyperparameters used for training the Dual-Head CNN model.

Inference Strategy

During test phase, the trained model predicts two probabilities: cube probability and sphere probability. The final probability is computed as:

$$p_{\text{final}} = p_{\text{cube}} \times p_{\text{sphere}}$$

The final class label is obtained using a threshold of 0.5:

$$\hat{y} = \begin{cases} 1, & \text{if } p_{\text{final}} > 0.5 \\ 0, & \text{otherwise} \end{cases}$$

we have also used Test-Time Augmentation (TTA). During TTA, each test image is evaluated multiple times using different transformations: original image, horizontally flipped image, image rotated by $+8°$, and image rotated by $-8°$.

Summary

- Hence, The proposed method uses a Dual-Head CNN to perform binary classification by separately predicting the presence of cube and sphere objects.
- The final probability is computed as the product of the two predicted probabilities.
- This design is useful because it forces the model to learn meaningful object-level features instead of directly learning only the final binary label.
- The use of Sobel edge pre-processing, coordinate augmentation, residual convolution blocks, dropout, data augmentation, and test-time augmentation improves the robustness and generalization ability of the model.

4.3 Vision Transformer (ViT)

Approach

A hybrid architecture that combines a **Vision Transformer (ViT)** with a **Slot Attention** module for binary image classification is used. The model is trained from scratch without using any pretrained weights.

The input image is first resized to 224×224 and divided into non-overlapping patches of size 16×16. Each patch is projected into a fixed-dimensional embedding vector. These patch embeddings are then processed using Transformer encoder blocks, which allow the model to learn global relationships between different regions of the image.

After the Transformer encoder, a Slot Attention module is applied. The purpose of Slot Attention is to group patch-level features into a small number of latent slots. Each slot can be interpreted as a learned representation of an object, region, or meaningful visual component in the image. Finally, the slots are averaged and passed through a classifier to predict the binary class label.

Motivation

A standard CNN mainly learns local features using convolution kernels. Although CNNs are effective for many image tasks, they may require deeper architectures to capture long-range relationships across the image. A Vision Transformer addresses this

limitation by treating an image as a sequence of patches and applying self-attention to model global dependencies between all patches.

However, a standard Vision Transformer usually aggregates all patch features into a single global representation. This may not explicitly separate different objects or important regions in the image. To address this, Slot Attention is added after the Transformer encoder. Slot Attention learns a fixed number of latent vectors, called slots, where each slot attends to different image regions.

Input Preprocessing and Data Augmentation

Each image is loaded in RGB format and resized/cropped to 224×224. The dataset is split into training and validation sets using a stratified split, where 15% of the training data is used for validation.

For the training set, the following data augmentations are applied:

- Random resized crop to 224×224
- Random horizontal flip with probability 0.5
- Color jitter for brightness, contrast, saturation, and hue
- Random rotation by up to 10°
- Random erasing with probability 0.25
- Normalization using ImageNet mean and standard deviation

For validation and testing, only resizing and normalization are applied.

Model Architecture

The proposed model consists of four main components:

1. Patch Embedding Layer
2. Transformer Encoder Blocks
3. Slot Attention Module
4. Classification Head

4.3.1 Patch Embedding Layer

The input image has size:

$$x \in \mathbb{R}^{3\times224\times224}$$

The image is divided into patches of size 16×16. Therefore, the number of patches is:

$$N = \left(\frac{224}{16}\right)^2 = 14^2 = 196$$

Each patch is projected into an embedding vector of dimension 192 using a convolution layer with kernel size 16 and stride 16.

Thus, the patch embedding layer converts the image into a sequence:

$$x_{\text{patch}} \in \mathbb{R}^{196\times192}$$

For a batch of size B, the shape becomes:

$$X \in \mathbb{R}^{B\times196\times192}$$

A learnable positional embedding is added to the patch embeddings:

$$X = X + E_{\text{pos}}$$

where:

$$E_{\text{pos}} \in \mathbb{R}^{1\times196\times192}$$

This positional embedding allows the model to retain spatial information about the original image patches.

4.3.2 Transformer Encoder

The patch embeddings are passed through 4 Transformer encoder blocks. Each Transformer block contains:

1. Layer Normalization
2. Multi-Head Self-Attention
3. Residual Connection
4. Layer Normalization
5. MLP Feed-Forward Network
6. Residual Connection

The self-attention operation allows every patch to attend to every other patch. This helps the model learn relationships between distant regions of the image.

The self-attention operation can be written as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

where Q, K, and V are query, key, and value matrices, and d is the embedding dimension per attention head.

The model uses:

$$\text{Embedding Dimension} = 192, \text{Number of Attention Heads} = 3$$

Therefore, the dimension per head is:

$$d_{\text{head}} = \frac{192}{3} = 64$$

The MLP hidden dimension is controlled by the MLP ratio:

$$d_{\text{MLP}} = 192 \times 4 = 768$$

After the Transformer encoder, the output still has the shape:

$$X_{\text{enc}} \in \mathbb{R}^{B \times 196 \times 192}$$

Slot Attention Module

The encoded patch features are passed into a Slot Attention module. Slot Attention converts the sequence of patch embeddings into a fixed number of slot vectors.

In this model:

$$K = 4$$

where K is the number of slots. Each slot has dimension:

$$D = 192$$

Therefore, the output of Slot Attention is:

$$S \in \mathbb{R}^{B \times 4 \times 192}$$

The Slot Attention module first initializes the slots using learnable parameters:

$$S_0 = \mu + \sigma\epsilon$$

The input patch features are projected into keys and values:

Attention logits are computed between slots and patch tokens

A softmax is applied over the slot dimension, so each patch token is softly assigned to the available slots. Then a weighted aggregation of the value vectors is computed to update each slot.

The slot update is performed using a GRU cell:

$$S^{t+1} = \mathrm{GRU}(U^t, S^t)$$

where U^t is the attention-weighted update vector. After the GRU update, an MLP with residual connection is applied:

$$S^{t+1} = S^{t+1} + \mathrm{MLP}(\mathrm{LayerNorm}(S^{t+1}))$$

The final slot representations summarize different important regions or components of the image.

Slot Pooling and Classification Head

After Slot Attention, the model obtains 4 slot vectors:

$$S = \{s_1, s_2, s_3, s_4\}$$

These slot vectors are averaged to obtain one global image representation:

The pooled representation is then normalized using Layer Normalization and passed through a classification head:

$$z' = \mathrm{LayerNorm}(z)$$

The classifier consists of:

1. Linear layer: $192 \to 192$
2. GELU activation
3. Dropout
4. Linear layer: $192 \to 2$

The final output is:

$$\hat{y}$$

where the two output logits correspond to class 0 and class 1.

Loss Function

Since this is a binary classification problem implemented with two output logits, the model uses cross-entropy loss:

The predicted class is obtained by taking the index with maximum probability:

$$\hat{y} = \arg\max_{c} \hat{p}_c$$

Training Strategy

The model is trained using the AdamW optimizer. AdamW is used because it combines adaptive learning rates with decoupled weight decay, which helps regularize Transformer-based models.

The learning rate is scheduled using Cosine Annealing.

4.4 Colour Space Analysis & Masking

Rather than treating the task as a pure representation learning problem, we have tested whether handcrafted or semi-handcrafted features derived from colour spaces and object segmentation masks could distinguish Class 0 from Class 1.

Colour Space Exploration:

- we have used RGB, HSV and LAB techniques.
- Inorder to understand the Feature Dynamics of color spaces we have tried out above exportimentation out of which we found that HSV and LAB did not perform good. RGB outperfromed the later two.
- For this data, RGB is found to be more accurate.

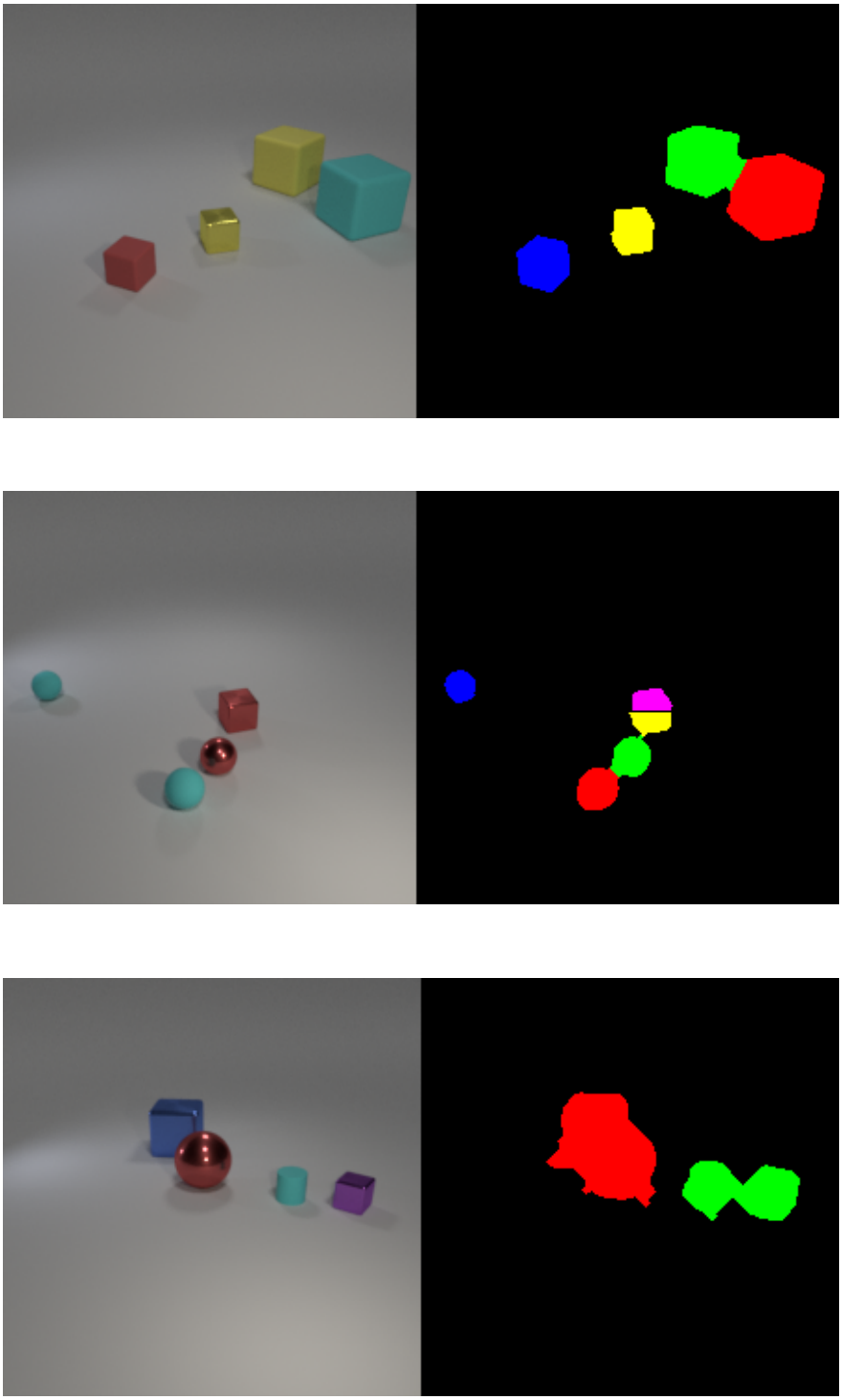
4.5 Object Masking

Object masking was explored to check whether isolating the foreground object regions could improve classification performance. The main idea behind masking is to suppress irrelevant background pixels and allow the model to focus more on the important visual regions, such as object shape, boundary, and colour distribution.

In this approach, a mask is generated to identify the probable object region in the

image. The mask separates foreground pixels from background pixels.

Object masking was useful for analysing whether the difference between Class 0 and Class 1 depends mainly on the object region or on the full image context. If the model performs well on masked images, it indicates that object-level visual such as shape, contour, and colour are important for classification. However, if masking removes useful contextual information or produces inaccurate masks, performance may decrease.



4.6 Validation Strategy

The validation set is created with 500 samples randomly from trainset

5 Experiments & Results

5.1 Experiment Comparison

Table 4: Summary of all experiments

#	Approach	Val Acc
1	Colour space + masking	0.753
2	ViT + ClevrTex	0.65
3	Slot Attention	0.752
4	**Dual CNN**	**0.7588**

5.2 Training Curves

Figure 2: Plot showing Training Loss Vs Epoch

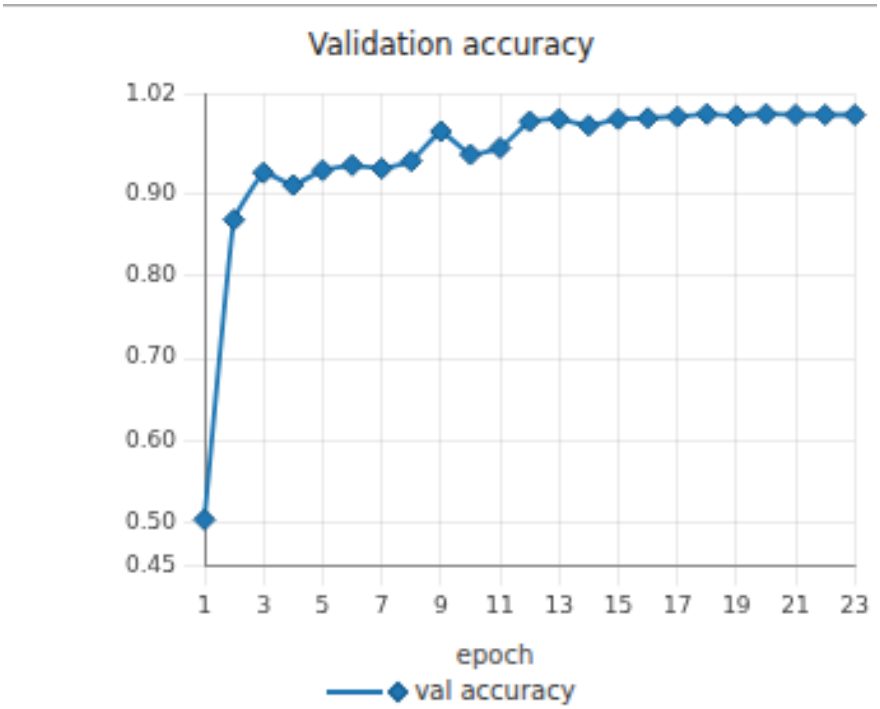


Figure 3: Plot showing Epoch Vs Validation Accuracy

5.3 Final Best Model

- Among all the models experimented with, the **Dual CNN architecture** gave the best performance.
- This architecture was more effective because it allowed the model to learn shape-specific information in a structured way. Instead of directly predicting the final binary class using a single output, the model used two separate prediction heads. One head focused on learning features related to the cube, while the other head focused on learning features related to the sphere.
- This design helped the model understand the visual components of the image more clearly.
- The shared convolution extracted useful low-level and high-level image features such as edges, boundaries, and shape patterns. These features were then passed to the two heads for shape-specific prediction.
- The final prediction was computed by combining the probabilities from both heads. Therefore, the model predicted the positive class only when both shape-related outputs were high. This made the model more reliable and reduced incorrect positive predictions when only one shape was strongly detected.
- Overall, the **Dual CNN model** was selected as the final best model because it achieved high accuracy and gave us a better architectural match for the task.

5.4 Challenges

- **Variable object sizes:** Objects within the same image were not uniformly sized. Smaller objects in the background are harder to classify by shape, particularly cylinders which can superficially resemble cubes or spheres depending on viewing angle and scale.
- **Object occlusion:** Some objects were partially hidden behind others. This meant the model could not always observe the full silhouette of each shape, making it difficult to reliably distinguish, for example, a partially occluded sphere from a cube.
- **Material reflectance ambiguity:** Metallic objects have specular highlights that alter their apparent shape, potentially confusing shape-based classifiers trained on matte-dominated examples.

5.5 Team Contributions

Table 5: Individual contributions

Member	Contribution
Harika	Vision Transformer (ViT) implementation; ClevrTex dataset exploration and integration,Hyper-parameter tuning
Deeba	Colour space analysis; exploratory data analysis and annotation; object masking pipeline
Rajat	Slot Attention implementation; Dual CNN design and training (best model); VAE+CNN

5.6 Reproducibility

- **Python version:** 3.10
- **Key libraries:** PyTorch 2.x, torchvision, pillow, numpy, scikit-learn
- **Hardware:** V100 GPU, 96 GB RAM
- **To reproduce:** Run `submission.py`. Ensure the dataset is placed in `/data/` with the original folder structure. The final cell writes `submission.csv`.
- **Random seed:** All random seeds set to 42 for reproducibility.

References

[1] Karazija, L., Laina, I., & Rupprecht, C. (2021). *ClevrTex: A Texture-Rich Benchmark for Unsupervised Multi-Object Segmentation*. NeurIPS 2021 Datasets and Benchmarks Track. https://www.robots.ox.ac.uk/~vgg/research/clevrtex/

[2] Locatello, F., Weissenborn, D., Unterthiner, T., et al. (2020). *Object-Centric Learning with Slot Attention*. NeurIPS 2020. https://arxiv.org/abs/2006.15055

[3] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR 2021. https://arxiv.org/abs/2010.11929

[4] Johnson, J., Hariharan, B., van der Maaten, L., et al. (2017). *CLEVR: A Diagnostic Dataset for Compositional Language and Elementary Visual Reasoning*. CVPR 2017. https://arxiv.org/abs/1612.06890